Project Name: SmartLoo

Team Members:

1. Farrel Emerson
2. Vannes Dharmawan
3. Waskitha Ghaziadiyata

Problem (The "Why")

- **Current Issue:**
  Toilet occupancy is currently difficult to monitor in real time. People may need to physically check whether a toilet is occupied, which can be inconvenient and inefficient, especially with multiple toilets. There is also no centralized way to know which toilets are currently available.
- **Impact:**
  This affects users who waste time checking occupied toilets and facility staff who have limited visibility of toilet usage. A software-based solution is needed to automatically detect occupancy and display the status online, making it easier to identify available toilets and manage facilities efficiently.

Proposed Solution

- **Our Approach:** SmartLoo will use ESP32 devices and limit switches to monitor the occupancy status of toilets across different floors and areas of the building. When a user locks a toilet door, the limit switch is triggered and the ESP32 sends the status to a Web API. The website then updates the toilet's status in real time.

  A central display screen located in the middle of the building will show the status of all monitored toilets, including **Available**, **Occupied**, and **Being Cleaned**. This allows users to quickly identify an available toilet without having to visit each floor or toilet individually.

- **Key Value:** SmartLoo reduces the time and effort required to find an available toilet by providing a centralized, real-time view of toilet occupancy. It also helps cleaning staff communicate when a toilet is unavailable due to maintenance or cleaning. By combining inexpensive ESP32 hardware with a Web API and website, the system can be expanded to support multiple toilets across multiple floors.

Core Features

List the 3 to 5 main features you will build. Remember: Quality over Quantity. It is better to have 3 perfectly written features with 100% test coverage than 10 buggy features.

- **Feature 1:** Real-Time Toilet Occupancy Detection

  Each toilet will use an ESP32 and limit switch to detect whether the toilet is **Available** or **Occupied**. The ESP32 will send the updated status to the Web API whenever the door is locked or unlocked.

- **Feature 2:** Centralized Toilet Status Display

  A website will display the current status of all monitored toilets across the building, organized by **floor and location**. A central screen will allow users to quickly see which toilets are available, occupied, or being cleaned.

- **Feature 3:** Toilet Management System

  Authorized staff will be able to manually update a toilet's status to **Being Cleaned** or **Available** through the web interface. This prevents users from being directed to toilets that are temporarily unavailable for cleaning or maintenance.

High-Level Architecture & Quality Assurance

Confirm your technical strategy here to ensure it meets the enterprise standards.

- **Architecture Pattern:** N-Tier, The system will be divided into separate layers: the ESP32/device layer, Web API/backend layer, database layer, and website/frontend layer. This separation makes the system easier to develop, test, and maintain.

- **Design Patterns Used:** Repository Pattern for handling database operations and Dependency Injection for managing dependencies between application components. These patterns will help keep the backend code organized and easier to test.
- **Code Quality Goal:** We commit to maintaining a green Jenkins pipeline, writing unit tests to be tracked via Cobertura, and keeping Qodana static analysis warnings below 10 (Target: 0).

Identified Challenges & Risks

Be honest. What parts of this project do you think will be the hardest to implement? (This is a great point for future discussions with the Senior Engineers).

- **Risk 1:** Reliable ESP32 and Limit Switch Detection, The limit switch needs to reliably detect when a toilet door is locked or unlocked. Incorrect installation, switch wear, or unstable connections could cause incorrect occupancy statuses.
- **Risk 2:** Keeping Toilet Status Consistent, The system needs to correctly handle different states such as **Available, Occupied, and Being Cleaned**. The backend must prevent conflicting updates, especially when an ESP32 automatically changes the status while staff are manually marking a toilet as being cleaned.